

MODAQ2 Documentation

Charles Candon

Casey Nichols

Robert Raye

August 11, 2026

Contents

MODAQ 2 Reference Design	3
Motivation	3
M2 Goals and Design Requirements	4
MODAQ 2 Versions	5
M2GO	5
M2 Hardware MODAQ M2 Hardware	6
Hardware Reference Design	6
Controller Considerations	6
Modular Architecture	6
Controller	7
Controller options that have been evaluated for M2.	7
Onlogic Karbon 410	7
ODROID H3	8
Raspberry Pi CM-5	8
Sensors, Instruments, and I/O Options	9
LabJack T8	9
BrainBoxes ED-582	9
Xsens MTi-G-710	10
Advanced Navigation GNSS Compass	10
Networking	10
M2 Software MODAQ M2 Software	11
Introduction	11
Ubuntu Operating System	11
Installing Ubuntu 22.04 on x86_64 Controllers	11
Installing Ubuntu 24.04 on Raspberry Pi CM5 (Arm64)	12
Why ROS2?	14
Install ROS2	14
Cloning the M2 Code Repository	15
Brief Description of the M2 RD Features and Functions	15
M2 Launch and Configuration Files	15
MCAP Bag Data Files	16
MODAQ Messages	17
M2 Supervisor	17
Labjack T8 ROS Package	17
M2 Control	18
Human-machine Interface	18
Headless Operation	18

Doxygen	18
Precision Time Protocol	18
what can M2GO do?	19
Getting Started	19
how do I get one?	19
M2 Github MODAQ Github Organization	20
Github Organization: MODAQ2	20
Building a MODAQ 2 System	20
Contributing to MODAQ2 with Github Submodules	20
M2 Technical Reference MODAQ M2 Tech Ref	22
ROS2 Nodes	22
Real-Time	24
Latency	24
Real-Time Linux	25
Validating RT Performance	25
Precision Time Protocol	25
Running ptp4l and phc2sys as System Services	26
Parsing MCAP Files	28
ADCs	30
4-20 mA Current Loops	30
Heading	32
M2 HMI MODAQ M2 HMI Example Design	33
Getting Started	33
nginx Configuration	34
ROSBridge Server	35
Web Client Framework	35
Basic HMI Design Elements	35
M2 Configurator MODAQ M2 Configurator Guide	39
Features	39
Getting Started	39
Recommended Setup	40
How to Use	40
Technical Details	41
Known Issues	42
Acknowledgements	43
Q&A MODAQ M2 Q&A	44
How does MODAQ differ from pre-built commercial options?	44
Why not just use generic DAQ controllers?	44
Is MODAQ plug-n-play?	44
What is M2GO?	45
Do I need to know programming to use MODAQ?	45
Which sensors or instruments should I use?	45
Can NLR help configure or customize MODAQ?	45
M2 Useful Links MODAQ M2 Links	46
MODAQ	46
ROS2	46
Ubuntu	46
C++	46
Git and Github	46

MODAQ 2 Reference Design

!!! note These documents are still under development. Please contact the open-water testing support team at NLR for more information. Contact Us

The National Laboratory of the Rockies' (NLR) first generation MODAQ (Modular Open-source Data Acquisition) is a highly performant, capable, and extensible platform for data acquisition, control, and automation that is based on the National Instruments cRIO controller line and family of expansion chassis and I/O modules. While MODAQ v1.0 (M1, hereafter) has proven itself both in the field and laboratory, several factors have indicated the need for additional solutions that were not well served on the M1 architecture, which led to the development of MODAQ 2 (M2). M2 is an approach based on generally available hardware and open-source development tools.

Motivation

Before diving too deeply into the details, it's important to understand the motivation behind M2. There were several drivers, but they all boil down to either:

1. We did not have a solution that satisfied particular project requirements, or
2. Feedback from customers and partners

In the case of #1, projects may have power or physical limitations that simply could not be met with any cRIO-based M1 variant. What was often the case is that we needed a solution that could be powered by batteries or otherwise have low power consumption. While our go-to cRIO (9049) in M1 is relatively power efficient (60W at maximum load), it's not practical for battery or power constrained applications. And despite its compactness, it can be too big (and heavy) for some use cases. Further, a cRIO was often overkill for some applications. While the cRIO scales up nicely, it does not scale well in the other direction.

With regards to feedback from the companies and organizations we frequently work with, there's often resistance to the M1 platform based on system cost or universality of the software development language- or both.

National Instruments publishes the price of its hardware on their website, so it's easy to get an idea of the price range for the controller and I/O modules. With strained budgets and an eye toward the future, where an organization may be considering arrays of devices or multiple device variants, it can be questionable if the M1 platform is a good fit, particularly when considering a pivot to commercialization.

Then there's the concern about the development language. M1 is written in LabVIEW, which is National Instruments' proprietary programming language. Here the issues are mostly around subscription costs and availability of LabVIEW developers. There's no sugar-coating that annual subscription costs for a LabVIEW Professional seat can be eye-opening- especially after bundling the Real-Time and FPGA modules.

Aside from the cost, availability of skilled LabVIEW developers pales in comparison to that of languages such as Python or C/C++. LabVIEW does not even appear in popular development language rankings. While it might not be fair to compare LabVIEW to general-purpose languages - and it's probably fairer to compare it to industrial programming languages such as those used for PLC or other vendor-specific packages - the fact remains that LabVIEW is rather niche and organizations with small teams find it challenging to find suitable engineering candidates that just happen to be competent in LabVIEW in the job market.

The intent here is not to bash on NI or LabVIEW, but rather to point out some of the challenges involved when collaborating with commercial enterprises- and even academia to some degree. We use NI and LabVIEW extensively at NLR and appreciate how they enable rapid development of data acquisition and control systems, complete with graphical user interfaces, for test setups. Some of us have experience in this ecosystem dating back to the 1900's!

M2 Goals and Design Requirements

There is little point to developing a new system without first establishing some overarching goals. Much of this can be gleaned from the previous motivation section, but here we explicitly state the goals we're trying to achieve with M2.

The primary goals of MODAQ 2 are to:

- Develop hardware and software architectures that can adapt to a wide range of missions and use-cases
- Assure measurement performance and quality parity with M1
- Utilize a development stack based on popular programming languages and tools
- Avoid vendor lock-in and single-source solutions

... and some Design Requirements:

- Modularity in both the design of the hardware and software
- Support a range of controller options and I/O types
- Ruggedized durability and 24/7 performance
- Precision time keeping and synchronization
- Real-time, deterministic performance
- Support for common communications protocols
- Configuration through a text-based file or utility
- Supervisory and automation control capability
- HMI and web-based data dashboard
- Built-in AWS S3 and SMTP support

MODAQ 2 Versions

The MODAQ 2 codebase has been developed to operate on controllers with either x86_64 (Intel/AMD64) or ARM64 CPU architectures. Initially we assumed there would be enough differences in these architectures to require developing and maintaining a codebase for each. However, the differences were few and easily addressed in the code. To keep things simple, there is a single unified version of the M2 software that supports both Intel and Arm architectures.

In the following Hardware and Software sections, guidance is provided on these topics:

1. Controller selection
2. Sensor and Instrument support
3. Hardware reference design specification
4. Operating system selection
5. Getting up and running with the ROS2 environment
6. M2 reference design software nodes and their usage

M2GO

[M2GO](#) is a prebuilt and pre-configured version of the M2 reference design using an Intel-based controller that NLR makes available for loan to qualifying 3rd parties. It's not a separate M2 version, rather it's just a physical build of the reference design described in the following sections of this documentation.

For parties interested in obtaining an M2GO loaner, please follow [this link](#) to contact us.

M2 Hardware MODAQ M2 Hardware

This section discusses the hardware supported by M2 and presents a reference design (RD) example that is supported by the M2 source code provided in the GitHub repository. It should be noted that any hardware mentioned has been either used in or evaluated for M2 projects, however this should not be interpreted as an endorsement or approval of any mentioned hardware product by NLR. Due to the general and flexible nature of the M2 software, M2 applications are not necessarily limited to these hardware selections.

!!! note

Reliability, durability, and suitability of hardware selections should be assessed prior to use in appl.

Hardware Reference Design

The purpose of providing this reference design is to offer a functional demonstration of the M2 platform. This hardware, along with the instructions in the [Software](#) section that follow, should yield the DIY user a properly functioning M2 build. Users seeking an “out of the box” experience, might consider our [M2GO](#) hardware loan program.

Since each project tends to be unique and measurement objectives can vary considerably, there is no typical configuration that will satisfy all situations. However, there’s usually significant overlap in base functionality that is common in most MODAQ projects, such as: measuring analog signals, motions, position, and temperatures. Advanced users can start with the RD and build upon it to create the necessary functionality—that’s what we do.

The following is a diagram of the M2 Reference Design:

!!! success “Equipment List”

Brand	Model	Qty	Description
OnLogic	Karbon 410	1	Controller
LabJack	T8	1	Multi-I/O
Netgear	GS305	1	Ethernet Switch
Adv. Nav.	GNSS Compass	1	Heading, GPS, PTP Server
Xsens	MTi-G-710	1	Inertial Sensor
BrainBoxes	ED-582	1	RTD to Ethernet
Samsung	T5	1	SSD Storage

The above M2 RD is a high-performance system designed for quality measurements. It’s capable of simultaneously sampling up to 8 analog channels at high-speed. The selected [T8 Multi-I/O interface](#) can be configured for a variety of input types and has both analog and digital outputs for control applications. Advanced features are included in the RD software such as: user HMI, email alerting, configuration utility, data uploading, and event logging.

Controller Considerations

Modular Architecture

M2 is designed with a modular approach that enables flexible hardware configurations and use of devices from various manufacturers and connection types.

Many of the devices supported in this RD communicate with the controller over ethernet (TCP/IP). Ethernet brings the following advantages:

1. Connection simplicity: Devices can be connected using standard networking components
2. PoE: Some devices may support Power over Ethernet, which further reduces the wiring complexity.

3. Flexible hardware placement: I/O modules that are TCP/IP capable can be moved closer to the measurement location, which reduces sensor/instrument wiring run lengths and the likelihood of signal corruption from noise sources. An M2 can be architected in zones or as a distributed system.

!!! note

Ethernet is non-deterministic and may not be suitable in time-critical applications that rely on precise timing.

Controller

M2 supports controller hardware based on either the x86_64 (e.g. Intel, AMD64) or Arm64 CPU architectures. Examples of such CPUs include the Intel Pentium, Atom, and Celeron series chips, as well as higher-powered variants in the Core series. For Arm64, only the Raspberry Pi CM5 has been evaluated to work with the provided M2 code. It's possible that other Arm64 based computers will work as well, though other devices have not yet been validated.

!!! warning Tests conducted with the Raspberry Pi Zero 2W were disappointing. It's suspected that the 512MB of SDRAM is the limiting factor, since all attempts to build the M2 code on the board failed. Building the code on another target and moving the executables to the Zero 2W was not tested.

Some considerations for CPU selection include:

- Power efficiency
- Performance expectations
- Number of processing cores and threads

The controller main board will include a preinstalled CPU. Some main boards may offer a CPU selection, however most do not. When selecting a main board, it's necessary to balance onboard features and specifications against the CPU capabilities.

Depending on the application, the main board might be sold as a Single Board Computer (SBC) or be prepackaged in an enclosure or case (e.g. industrial PC). It's recommended to avoid general-purpose consumer oriented boards and instead look for boards targeted for industrial or embedded applications. Some devices marketed for 'Edge' computing may also be suitable. Desirable features in a main board might include:

- Minimum of 4 GB RAM
- M.2 slots that support NVMe (or SATA) SSD storage
- Multiple USB ports (3.0 or greater preferred)
- At least one gigabit (or better) ethernet port
- Precision Time Protocol (PTP, IEEE 1588v2) support in ethernet controller (e.g. Intel i210)
- Real Time Clock (RTC) with battery backup
- Serial communications port (RS-232/RS-485/UART)
- Wide input voltage tolerances
- Video display interface (optional)

Depending on the operating environment, it might be desirable to select a model that is rated for a wide operational temperature and humidity range.

!!! note

The above features are preferred, but not mandatory. Individual use-cases and performance requirements may vary.

Controller options that have been evaluated for M2.

Onlogic Karbon 410

Karbon 410 While there are many vendors that sell ready-made rugged, industrial computers, we selected the relatively low-cost and modestly equipped Karbon 410 from Onlogic for evaluation as a mid-range M2 controller.

The Karbon 410 is a fanless design and comes in a heavy-duty rail-mountable enclosure with a large passive heatsink. Specification-wise, it includes all of the desirable features listed in the previous section and can be optioned with either GPIO or two serial ports (RS-232/422/485).

Some notable features of the K410:

- Choice of either a dual or quad core Atom CPU
- Intel i210 ethernet controllers support PTP
- 9-48 VDC input power range
- Carries numerous regulatory certifications, including for shock and vibration
- Full-stress power draw (when equipped with Atom x6211E CPU) is <15W and can deep sleep «1W

ODROID H3

ODROID H3 The ODROID H3 is a very low cost fanless single board computer with a quad core Celeron N5105 CPU. It's a basic, moderately powered performer that lacks the ruggedization and certification bonafides of the K410, however it's versatile and well equipped.

Some notes on the H3:

- Small 110x110mm footprint
- Dual SODIMM DDR4 RAM slots and one NVMe m.2 slot
- PTP support on the Realtek RTL8125B ethernet controllers requires some effort and not as robust as the Intel ethernet controllers.¹
- Supports input power in the range of 14-20 VDC, which is a little unusual.
- Full stress power consumption is ~18W and <1W in suspend mode

Raspberry Pi CM-5

Raspberry Pi claims that approximately 75% of its sales are to the Industrial and Embedded sector and their devices are found powering a variety of devices in assorted applications that are similar or adjacent to MODAQ. Robustness and dependability are important to us, and is one of our stated design requirements, therefore we're cautiously optimistic when it comes to the CM5. Our experience thus far has been positive, however users are encouraged to perform their own evaluations and weigh the risks.

CM5 The CM5 is a credit-card sized microcomputer that needs to be inserted into a carrier or expansion board in order to expose the I/O, communications, storage, and power connections. This can increase the overall footprint and power draw, but since the CM5 is basically useless without a carrier board, they must be assessed together when evaluating options. In our tests, the CM5 with the OEM carrier board power draw under MODAQ operation was measured around 3.4 watts (NOTE: this power consumption value is just for the controller, not including any external peripherals such as the LabJack T8).

Notable features:

- NVMe support on the OEM carrier board
- Hardware PTP support
- Onboard RTC with battery backup
- Compatible with Raspberry Pi HATs
- 3rd party carrier boards and industrial housings available for greater flexibility

Notable disappointments:

- No low power idle mode or hibernation/sleep
- OEM carrier board is rather large (160 x 90mm) and only has one ethernet port

¹Using the minified version of any of the required JS libraries is totally optional. We prefer minified, since it's a smaller overall payload and can improve performance. The disadvantage to using minified libraries is that all unnecessary whitespace characters are removed, as are comments, and some optimizations may be applied (i.e. variable name shortening), so inspecting or editing the library code can be very difficult- even though it's still in plain text.

- Published MTBF (Mean Time Between Failure) under high stress environmental conditions is 16k hours (~2 years at 24/7 duty). MTBF is closer to 10 years in stable, controlled conditions. (to be fair, we could not find published MTBF for the Karbon 410 nor the ODROID H3. The Karbon 410 does meet a variety of IEC and MIL-STD shock and vibration standards)

Sensors, Instruments, and I/O Options

The M2 Reference Design includes support for the following devices:

- LabJack T8
- Xsens MTi-G-710 GNSS/INS
- Advanced Navigation GNSS Compass
- Brainboxes ED-582

These provide the following capabilities:

- ± 10 VDC analog input and output
- 4-20 mA current loops
- Logic level digital input and output (also known as TTL or DIO)
- Position, heading, velocity, acceleration, rotation, and orientation
- IEEE-1588v2 PTP time server
- Temperature measurements using PT100 or PT1000 RTD sensors

!!! note

The devices listed will work on all controllers mentioned in the previous section.

M2 supports considerably more hardware options than are included in the Reference Design distribution. Contact us to learn more.

LabJack T8

Labjack T8

Figure 1: Labjack T8

The LabJack T8 provides a lot of useful I/O in a single highly performant ethernet or USB connected device.

Some highlights:

- 8 simultaneous sampling 24-bit analog to digital converters (ADC), with channel-to-channel isolation and sampling up to 40 kHz
- Up to 20 channels of digital I/O (DIO)
- USB or ethernet connectivity- can also be powered by USB or PoE
- Available option for 14-bit ± 10 VDC digital to analog (DAC) output
- Available option for 4-20 mA current loop input

The Reference design includes M2 software nodes to stream the ADC channels at up to 40 kHz or read the channels on demand. It also includes nodes to command an output voltage to the DAC module and read/write to the DIO.

BrainBoxes ED-582

ED-582

Figure 2: ED-582

The ED-582 is a simple solution for acquiring up to 4 RTD measurements over an ethernet connection. Each channel can be configured independently for either PT100 or PT1000 temperature sensors and their associated temperature coefficients.

It has 16-bit resolution and can acquire at up to 5 samples per second- which provides just over 1 Hz sample rate with all 4 channels enabled.

Xsens MTi-G-710

MTi-G-710 The world of inertial sensors has a very broad price, performance, accuracy, and quality spectrum, ranging from cheap hobby grade to high-end, export-controlled devices costing many 10's of thousands of dollars. We tend to specify 'moderately' priced models such as the Xsens MTi-G-710 in many projects.

This device uses a sensor fusion engine to develop 3D orientation solutions using GNSS satellite signals and built-in 3-axis accelerometers, gyros, and magnetometers.

Advanced Navigation GNSS Compass

The M2 reference design includes support for the GNSS Compass from Advanced Navigation. This ethernet connected, PoE powered device includes a GNSS satellite receiver with RTK support, built-in dual antennas, and a PTP time server.

This device is a compliment to the Xsens Mti-G-710, since the GNSS Compass, with it's dual antennas can provide a superior heading estimate and with an RTK correction signal available, can provide can provide superior positioning estimates. In addition, it can provide an estimate of heave and provides a highly accurate time reference for the M2 system.

Networking

We purposely designed the RD with simple, basic networking. On the plus side, this is cheap and requires no special configuration. It's also sufficiently fast for the selected components. However, in more elaborate builds, especially those with multiple PTP capable devices, a managed switch with PTP support will be more desirable.

The specified switch does not have have a router, so all devices will need to configured with a static IP address all on the same subnet. A laptop with an IP on the same subnet could be attached to one of the unused ethernet ports to access the [HMI](#).

M2 Software MODAQ M2 Software

This section will discuss the software architecture of M2 and explain the reference design available on the NLR GitHub at MODAQ2. For discussion on design decisions and details on technical aspects of the software design, please see the Technical Reference page.

Introduction

M2 leverages the ROS2 ecosystem of libraries and tools to provide a well established way of developing a data acquisition system suitable for a variety of projects. ROS2 was selected because it is easy to learn, has a robust support community, completely free and open-source, supports multiple programming languages, and is reliable for long term use in industrial settings. There is also a plethora of existing ROS packages that users can leverage in addition to the packages we developed for MODAQ 2.

The MODAQ 2 project is an amalgamation of ROS packages, tools, and guidance for developing data acquisition and control applications for marine energy devices using a ROS based architecture. The MODAQ 2 Reference Design includes several packages that are targeted for laboratory and field applications and allow most developers with a basic background in programming to spin up a high-quality DAQ and control system. While we say ‘for marine energy devices’, that points to M2’s origin (and some of its instrument support) and not necessary its exclusive domain. M2 can be used in a wide variety of applications that need a performant and reliable data acquisition solution.

The main software aspects of MODAQ 2 are as follows:

- Ubuntu Linux Operating System
- ROS2 Humble Hawksbill (or Jazzy Jalisco for Arm64)
- Community Developed ROS Packages
- NLR Custom Developed ROS Packages
- Third Party DAQ Driver libraries

Ubuntu Operating System

Ubuntu is a free to use² Linux distribution that has Tier 1 support³ by the ROS ecosystem. To get started working in Ubuntu, it is recommended to visit the [Hardware page](#) of this document to ensure that your controller meets the requirements of the MODAQ 2 design. After that, you can follow the instructions provided below to install the correct version of Ubuntu Desktop on your controller. At the present time, select Ubuntu 22.04 Desktop for x86_64 (Intel and AMD64) controllers and Ubuntu 24.04 for Arm64.

!!! note Ubuntu 24.04 officially supports the Raspberry Pi CM5 hardware. Since ROS2 releases tend to be tied to specific Ubuntu versions, Arm64 applications will need to use ROS2 Jazzy instead of Humble.

M2 was originally written for x86_64 controllers with Ubuntu 22.04 and ROS2 Humble. We have yet to vali

Installing Ubuntu 22.04 on x86_64 Controllers

The easiest way to install Ubuntu on a x86_64 controller is to create the installer on a USB stick.

1. Download either the Desktop or Server install image here.

!!! question “Ubuntu Desktop or Server?” Either Ubuntu Desktop or Server edition can be used with M2.

²Using the minified version of any of the required JS libraries is totally optional. We prefer minified, since it’s a smaller overall payload and can improve performance. The disadvantage to using minified libraries is that all unnecessary whitespace characters are removed, as are comments, and some optimizations may be applied (i.e. variable name shortening), so inspecting or editing the library code can be very difficult- even though it’s still in plain text.

³We’re using chart.js v3.3.2 since at the time we were creating the HMI there were compatibility issues with the plugin-streaming library on v4 and higher. That may have been resolved in later releases of plugin-streaming.

****Choose Desktop if:**** you might want to connect a keyboard, mouse, and monitor to the controller

****Choose Server if:**** you are comfortable working in the command line interface (AKA terminal/console).

Desktop and Server equally support remote development (such as <https://code.visualstudio.com/remote>).

2. Install a USB writer utility, such as BalenaEtcher, which is available for Windows, Linux, and MacOS hosts. (NOTE: Ubuntu has a creation tool built in: Startup Disk Creator. So you can skip this step if you're using a computer with Ubuntu to set up the controller)
3. Burn/etch the Ubuntu 22.04 file downloaded in Step 1 to a blank USB stick.
4. Place the newly etched USB stick in a USB port on the controller target and boot from the stick.

!!! note Most x86_64 based controllers should be set to automatically boot from USB media, however if this does not happen, it may be necessary to go into BIOS and change the boot order so that the USB ports are prioritized. This process varies from manufacturer to manufacturer, so consult documentation for your controller if unsure how to do this.

5. Follow onscreen instructions to install the OS to the desired location on the controller.

Installing Ubuntu 24.04 on Raspberry Pi CM5 (Arm64)⁴

There are a couple methods of installing an OS to a CM5, which to choose will depend on the version of the CM5 you have and where you want the OS to be installed. We chose CM5s with onboard eMMC storage and installed the OS there. We could have opted to install the OS to the NVMe SSD drive, however we prefer to use the SSD for data only. If the OS and data reside on the same drive, it's more involved to swap the drive (for instance, it might be desirable to quickly replace a full drive with an empty one in the field), since the OS will need to be cloned to the new drive. Lite variants of the CM5 (those without eMMC) could have the OS installed to a microSD card.

Download the Ubuntu 24.04 Desktop install image [here](#) or Server image [here](#).

!!! question "Ubuntu Desktop or Server?" Either Ubuntu Desktop or Server edition can be used with M2.

****Choose Desktop if:**** you might want to connect a keyboard, mouse, and monitor to the controller and r

****Choose Server if:**** you are comfortable working in the command line interface (AKA terminal/console).

Desktop and Server equally support remote development (such as <https://code.visualstudio.com/remote>).

Install the Raspberry Pi Imager on the Host Computer Following the installation instructions found [here](#).

Install Ubuntu on microSD Card (CM5 Lite only) It's recommended to follow the instructions in the official Raspberry Pi Documentation. In Step 2, do one of the following (not both!):

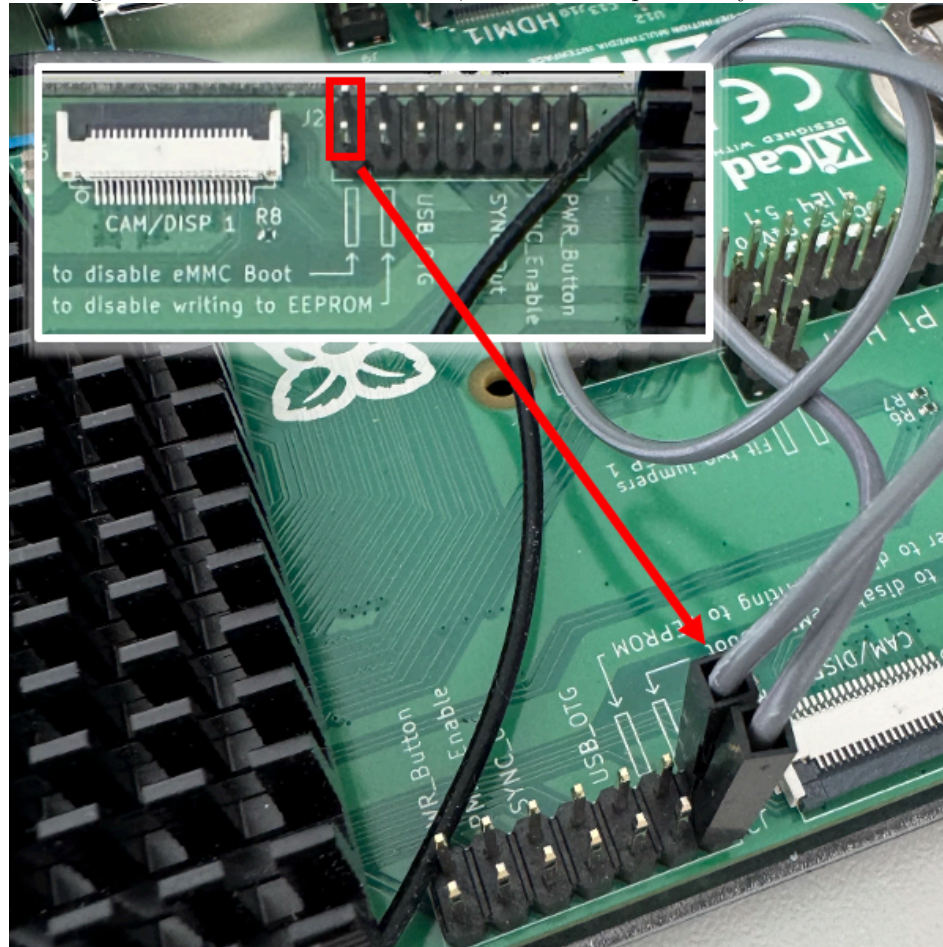
1. Download the Ubuntu 24.04 install image from the link above. Scroll down to "Use custom" in the Raspberry Pi Imager software. In the file dialog that pops up, select the image you downloaded.
2. Scroll down to "Other general-purpose OS" in the Raspberry Pi Imager software. On the next screen, select Ubuntu, then Ubuntu Desktop 24.04.4 LTS (64-bit). The imager software will download the OS.

The final step is to select the destination for the software to write the OS image, per the documentation.

⁴Since finding the dependencies as minified libraries in the correct version can be a bit of a pain, here are direct links: luxon v3.4.40 and chartjs-adaptor-luxon v1.0.0

Install Ubuntu to eMMC or SSD drive This is a bit more involved and borderline hacky, but this is how it's done! NOTE: this method can be used with a CM5 Lite for installing the OS to an NVMe/SSD instead of microSD (Lite models do not have eMMC).

1. Download the desired Ubuntu 24.04 image per links above (optionally, the image can be selected and downloaded in the Raspberry Pi Imager software during Step 8).
2. Install the USB Boot utility on your host computer per the instructions found here.
3. Place the CM5 in USB Boot Mode. If using the OEM CM5 carrier board, this is accomplished by



jumpering 2 pins on the carrier board:

If using a 3rd party carrier board, consult the instructions for that board to place the CM5 in USB Boot Mode.

4. Connect the USB-C connector on the carrier board to your host computer
5. Make sure rpiboot that was installed in Step 2 is running. The CM5 should now be mounted.
6. Launch the Raspberry Pi Imager software
7. In the Imager software, select RASPBERRY PI 5 as the **Raspberry Pi Device**.
8. Under **Operating System**, do one of the following (not both!):
 1. Download the Ubuntu 24.04 install image from the link above. Scroll down to "Use custom". In the file dialog that pops up, select the image you downloaded.
 2. Scroll down to "Other general-purpose OS". On the next screen, select Ubuntu, then Ubuntu Desktop (or Server) 24.04.4 LTS (64-bit). The imager software will download the OS. (NOTE: It's possible that a later version of Ubuntu 24.04 LTS will appear in the Imager options. This is okay, simply select the latest version that starts with 24.04)
9. Under **Storage**, find the CM5 target where the OS is to be installed (either the eMMC or NVMe SSD).
 1. eMMC - There should be an option with wording similar to this: "mmcblk0 Raspberry Pi multi-

- function USB device”. The important part is the “mmcblk0”, where “mmc” is the eMMC.
2. NVMe SSD - There should be an option with wording similar to this: “nvme0n1 Raspberry Pi multi-function USB device”. The important part is the “nvme0n1”, where “nvme” is the NVMe SSD.
 10. Proceed to flash the OS to the selected target. **WARNING:** this step will erase the target. Make sure the target is either blank or contains nothing of value since it will be deleted.
 11. Remove the jumper installed in Step 3 and apply power to the carrier board. The CM5 should now boot into Ubuntu.

Why ROS2?

The Robotic Operating System (ROS) is a collection of libraries, tools, and middleware that were designed to benefit the robotics development community. The qualities of ROS that benefit the robotics community also benefit the data acquisition and control for marine energy community. It includes native functionality, such as concurrent processes, interprocess communications, saving data, and runtime parameter loading, which reduces development load. It provides an easy entry point for software development for data acquisition control but it also is very scalable and has the capabilities to do very advanced computational tasks if needed for a project. For example, after installing ROS2 and MODAQ 2 packages, you can quickly run a process that collects and logs sensor data in mcap bag files⁵ for data analysis. However, if you need to build code that is deterministic or [real-time](#), this is also possible with ROS but requires a much lower level understanding of computer programming and is likely not necessary for many applications. In summary, ROS is adaptable and extensible to meet the requirements of a data acquisition and control project.

!!! note Unless otherwise noted, wherever we say ROS, we mean ROS2. There are significant differences between ROS1 and ROS2 and they are not directly compatible. Use caution when searching for ROS online, a lot of links often go to ROS1 resources.

The fundamentals of ROS are covered extensively in many online trainings, videos, whitepapers and classes, and we have collected our recommended starting places in the [Useful Links](#) page.

For MODAQ 2 usage, the main concepts to understand are as follows:

- Nodes: the software processes that make up a ROS system
- Packages: collections of nodes usually including all the necessary nodes to use a system component
- Topics, Messages, Publishers and Subscribers: how ROS nodes communicate with each other
- Middleware DDS: the background system that allows nodes to communicate reliably and with modularity
- Command Line Tools: useful tools to test and debug ROS systems
- Build System: colcon is used to build ROS2 packages
- Launch Files: files that define the startup and execution of a complete ROS system
- Parameters: configuration items that can modify how the ROS system operates
- Bag (mcap) files: data files that database the information collected and communicated through ROS

Install ROS2

It is recommended to follow the step-by-step process available from [ros.org](#) to properly install ROS2 on the linux controller target.

For x86_64, AMD64 targets using Ubuntu 22.04.x, install Humble Hawksbill.

For ARM64 targets using Ubuntu 24.04.x, install Jazzy Jalisco.

The following command will verify the proper installation of the main components of ROS on your Ubuntu 22.04 controller. This doesn't include the [Labjack LJM Library](#) which is required for using the Labjack T8 device. Instructions for installing this are described below.

⁵Replace 'localhost' with the IP address of the M2 controller hosting the HMI webpage.

```
## Please follow the provided instructions for installing ROS2 but you can verify the install with this
sudo apt update
sudo apt upgrade
sudo apt install ros-humble-desktop ros-dev-tools ros-humble-rosbag2 ros-humble-rosbag2-storage-mcap ng
```

Cloning the M2 Code Repository

Follow the instructions described in [M2 Github](#).

Brief Description of the M2 RD Features and Functions

The M2 RD (and M2GO) ‘ships’ with several capabilities and I/O support from the larger M2 codebase. We chose to limit the RD to basic, core functions that are common across most M2 builds as a useful technology demonstration. If a needed feature or input device is not found in the RD, chances are, it’s available in the full code. Contact the development team for more information.

M2 Launch and Configuration Files

M2 utilizes the launch and configuration features available in ROS2. The launch file contains the instructions to easily start the M2 application with a single command while the config file contains parameters that are passed to specified nodes at runtime. During development of M2, we tried to make M2 nodes as stateless as possible. This means we avoided hardcoding values that are likely to change, such as the IP address of a device or the email addresses for sending alerts. Instead, these can be specified in the config file.

The m2_launch package cloned from the repo includes the python launch file which starts all the parallel processes (nodes) of the MODAQ 2 system. In m2_launch.py, the nodes of the system are specified and the config file is supplied. All ROS parameters of a node should be specified in the config file.

!!! note The name of the node specified in the launch file must match the node name used in the yaml config file. I.e. M2Supervisor is specified in both in the launch file and the config yaml file.

Example launch and config files:

```
# Launch File
import yaml
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch_ros.actions.node import ExecuteProcess
from datetime import datetime
from ament_index_python.packages import get_package_share_directory

##### PATH TO MODAQ CONFIG FILE #####
## FYI THIS PULLS THE FILE FROM THE INSTALL DIRECTORY, NOT THE SRC DIRECTORY
config = os.path.join(
    get_package_share_directory('m2_launch'),
    'config',
    'm2_config.yaml'
)

def generate_launch_description():

    return LaunchDescription([
        Node(
            package = "m2_supervisor",
            executable = "m2_supervisor_node",
```

```

    name = "M2Supervisor",
    parameters = [config],
    output="screen",
    respawn = True
),
Node(...other nodes)
)]

# Config File
M2Supervisor:
  ros__parameters:
    loggerPath: "/home/m2/Log"
    loggerLimitBytes: 1000000
    emailSendAddr: "youreemailwithsmtpenabled@email.com"
    emailSendPwd: "passwordhere"
    emailGroup1:
      - "youreemail@email.com"
    emailGroup2:
      - "youreemail2@email.com"
    analyzedTopics:
      - /ain_flap
      - /T8din
    topicErrorEmailSnooze: 60

```

The launch file and config files are moved once the build process is completed. To access these files, you must run `source install/setup.bash` in terminal. You should then be able to launch the file with the command `ros2 launch m2_launch m2_launch.py`. Executing this will start all the ROS nodes together. If this fails, errors will be reported in the terminal to assist you with troubleshooting.

MCAP Bag Data Files

The data collected by different nodes within MODAQ 2 is published on specific topics. This enables the data to be recorded and saved for future use. This is implemented by using the ROS2 bag recorder node and the mcap storage method. The ROS2 bag package can be installed by running `sudo apt install ros-humble-rosbag2 ros-humble-rosbag2-storage-mcap` in terminal. This is a useful tool for debugging robotic applications however, it is designed with enough reliability to function as a robust data recorder for data acquisition applications. The `bag_recorded` package in the MODAQ 2 reference design takes the rosbag2 API and adds some flexibility to it to allow for recording only specified topics as well as the ability to start and stop the recording programmatically.

Configuring Bag Recording The `bag_recorder_node` is configured in the launch file configuration yaml file and includes the folder in which to save the bag files (mcap files). It includes the length of the files in seconds. It is best to break up the mcap files so they don't become too large and to reduce to possibility of corruption if the system crashes or power is lost. It also includes the logged topics. All topics in this list will be included in the bag files. You can also enable recording all topics by specifying "*" as the logged topics.

```

BagRecorder:
  ros__parameters:
    dataFolder: "/home/m2/Data"
    fileDuration: 60
    loggedTopics:
      - /system_messenger
      - /ain
      - /do
      - /T8din

```

MODAQ Messages

`modaq_messages` is a custom message specifier that is used in MODAQ 2. ROS includes many message types by default but it was deemed necessary to build our own custom message types more applicable to our application of data acquisition. This ROS package has no nodes in it but it includes the CMake commands to turn the `msg` files into headers which can be referenced in different nodes. For example, `/system_messenger` uses `modaq_messages/msg/systemmsg.hpp` to define the type in different nodes. To use these custom messages, they must be included in the `package.xml` and the `CMakeLists.txt` file.

`CMakeLists.txt`:

```
find_packages(modaq_messages REQUIRED)
ament_target_dependencies(m2_supervisor_node rclcpp std_msgs modaq_messages rosbag2_cpp)
```

`package.xml`:

```
<depend>modaq_messages</depends>
```

M2 Supervisor

The `m2_supervisor` is a package that serves as the central process of MODAQ and includes some functionality that users may find valuable. It includes features for logging events, sending emails and monitoring the health of the bag files being recorded.

Logger The logger processes `/system_messenger` messages that have the log switch turned on and stores the information into a human-readable text file for diagnostics during operations.

A rate-limiting feature called the “snoozer” can be enabled in messages posted to `/system_messenger` to prevent flooding the log with repeated message occurrences. This will suppress duplicate message writes over the selected time interval. For instance, if a 100 Hz loop encounters an error that publishes to `/system_messenger`, the snoozer will prevent 99 duplicate messages per second from getting written to the system log. If the messages continue after the snoozer expires (e.g. the error condition persists), the snoozer will allow another write of that message to the system log and then snooze again for the set interval time.

Emailer The emailer processes `/system_messenger` messages that have the email switch turned on and sends an email with `smtp` which contains the important information in a human readable format. For this to work, you must have an email with `smtp` enabled - this is possible with a free gmail account.

Up to two distribution lists are supported in this release of the Emailer and those lists can be populated in the config file. Each list can contain one or more email addresses, however there’s going to be a lengthy list of recipients, it’s suggested to create a group (using something like google groups) so that the logger only sends out one email and that gets distributed to the group.

Distribution list 1 is always enabled if emailing is selected for the alert topic. If distribution list 2 is enabled, both list 1 and list 2 will receive the message. Best practice for this system is list 1 is used for developers or maintainers to receive critical or diagnostic system related messages, while list 2 is would be used for more general status updates.

The snoozer feature described in the logger section also applies to emails. If enabled, it will suppress repeated messages over the preset interval to prevent email floods.

BagAnalyzer Coming soon! This feature is still under development. The goal is to develop a process that monitors bag recording to ensure that all topics are being recorded as expected.

Labjack T8 ROS Package

The `labjack_t8_ros2` package was developed specifically for use in MODAQ 2. This package relies on a third party driver to interface with the Labjack T8.

Labjack LJM Library MODAQ 2 Reference Design includes the Labjack T8 to make high-speed measurements up to 40 kHz. We have provided the `labjack_t8_ros2` package to connected to T8 devices and stream data to M2 so it can be acted upon with control rules and/or logged as measurement data for post-processing and analysis. To use the `labjack_t8_ros2` package, the Labjack LJM library needs to be installed on your controller.

The instructions for installing this library are in `{MODAQ 2 Repo}/src/labjack_t8_ros2/README.md`

M2 Control

Currently the `m2_control` package just handles the interactions between the system and the HMI. The ROS2 ecosystem is well suited for more advanced control operations. A simple control algorithm could be implemented by creating a subscriber for the control rule input (i.e. the measurement of a battery voltage) with a callback class function. In the callback function, you could implement your algorithm (i.e. if voltage is too low, turn off a pump). The control could be actuated by then publishing a command to a digital output to turn something on or off (i.e. turning off the pump).

ROS2 also has built in control algorithms that you may find useful despite their main focus being on robot control. See the package `ros2_control` to learn about the advanced control options developed for use in ROS2.

Human-machine Interface

The HMI is described on the [HMI page](#)

Headless Operation

MODAQ 2 will often need to run headless or automatically start on the boot of the controller. This can be done with linux services.

Instructions for putting this in place can be found in `{MODAQ 2 Repo}/service/README.md`

Doxygen

The development of c++ documentation has started and is a work in progress. To view the c++ documentation made using doxygen, navigate to the docs folder and build using the bash command `doxygen`. More information is available in `docs/README.md`. Once built, the doxygen documentation can be viewed on the nginx webserver by opening `{M2_Controller_IP}/index.html`

Precision Time Protocol

The precision time protocol or IEEE1588v2 or PTPv2 is a protocol used for synchronizing clocks on a network. While this can be done at the software level on the linux kernel, it is recommended to use a hardware implementation for synchronization in the microseconds. To do this, the network interface card (NIC) of your controller must officially support PTP. This can be determined with the model number of the NIC which can be determined with the command `lspci | grep Ethernet` which returns:

```
01:00.0 Ethernet controller: Intel Corporation I210 Gigabit Network Connection (rev 03)
03:00.0 Ethernet controller: Intel Corporation Ethernet Controller (2) I225-IT (rev 03)
```

Both of these Intel ethernet NICs (I210 and I225) support PTP. We recommend purchasing controllers with Intel NICs as they have the highest reliability when using PTP.

M2GO is a physical representation of the M2 Reference Design discussed in the previous sections. It comes preconfigured, tested, and ready to go.

what can M2GO do?

Getting Started

how do I get one?

M2 Github MODAQ Github Organization

This section will discuss the layout of the MODAQ 2 codebase on Github.

Github Organization: MODAQ2

Because MODAQ 2 is designed to be modular, it was laid out using an organization in Github which can be found at this link: [MODAQ2](#). This enables the ability to clone or copy the necessary modules for your specific implementation of MODAQ 2. The main reason of taking this approach to software organization is to streamline the set up of a new system and streamline contributing to the open-source MODAQ 2 project.

Building a MODAQ 2 System

It is recommended to utilize git and Github/Gitlab for building a MODAQ 2 system to efficiently track changes and maintain version control. We recommend initializing a new repo on your Github account and bringing in the necessary modules for your project. You can also add any modules you may need that currently aren't available from the MODAQ2 organization. These are the steps to do this:

1. Login or make an account on [github.com](#)
2. Make a new repository on your account - we recommend including the ROS .gitignore template
3. Start your development computer and navigate to the folder you want to keep your repo in. I.e. `cd ~`
4. Clone the repository you just created to your development computer using `git clone https://github.com/{your_account}/{your_repo}`
5. Move into your repository and create a src directory using `cd {your_repo}` and `mkdir src`
6. Clone the MODAQ2 repositories you need into the src folder with `git clone https://github.com/MODAQ2/{repos_you_need}`
7. To be able to push your build to your new github repository, you will need to delete the .git folder from the repos you have cloned. `cd m2_core` and `sudo rm .git -r`
8. You can now commit and push your code stack to github and it will be detached from the MODAQ2 organization
9. Please consider contributing to the MODAQ2 organization if you have changes the community can benefit from. This process is described below.

!!! note You will likely need to set up a personal access token to clone and push to your github repo. Visit this link to learn how to do that. [Link](#)

Contributing to MODAQ2 with Github Submodules

When contributing to the MODAQ2 organization, it is recommended to fork the MODAQ2 repos you are using and bring in them into your main repository as git submodules. This is essentially adding a git repo inside another git repo. The changes tracked in the submodule are actually relative to the repo of the submodule rather than the broader repo. Say you make a change to the labjack_t8_ros2 package that would be useful for everyone who uses labjacks, you can make a pull request on the labjack_t8_ros2 repo and your changes could help out all users of MODAQ 2. This meets our project requirement of maximizing modularity for MODAQ 2 users.

All of the current packages/modules for MODAQ 2 can be found on the Github Organization.

To fork the repositories you are using, follow these steps:

1. Login or make an account on [github.com](#)
2. Navigate in your browser to the MODAQ2 package you are wanting to contribute to
3. Click the fork button in the top right corner
4. Navigate to your personal fork of the repo. You can find it on your github account in your repositories
5. Get the URL of your personal fork. I.e. `https://github.com/{your_account}/m2_core.git`

The recommended process for getting started with submodules is to start a top level repo which will hold the submodules:

```
cd #go home
mkdir MODAQ2-Dev #this directory should be named based on your project name
cd MODAQ2-Dev
git init #init the top level repo
mkdir src #all ros2 packages should be in the src directory
cd src
git submodule add https://github.com/{your_account}/m2_core.git #this is the main module for MODAQ 2
git submodule add {Other MODAQ 2 packages/modules}
```

Now your fork of m2_core is a submodule in your MODAQ2-Dev repo. Changes you make to m2_core will be tracked locally to the m2_core forked repo.

To contribute your changes, push to your forked repo and then open a pull request to the repo on MODAQ2 on Github.com

More information on forking, submodules and pull request can be found in the [Useful Links](#)

M2 Technical Reference MODAQ M2 Tech Ref

This section contains details on some of the core M2 functions and nodes, as well as deeper discussions on some of the technical considerations.

ROS2 Nodes

The M2 Reference Design includes the following nodes. Click on the item to expand its description.

Node: `m2_supervisor_node`

This node manages a number of MODAQ's core functionality, which include system logger, email alerts, and rate limiter (snoozer) Package: `m2_supervisor` Alias: `M2Supervisor` Configuration Parameters:

log file path

log file size limit

SMTP settings

distribution email lists (2x)

analyzed topics

email snooze interval

Publishers: `/system_messenger` Subscriptions: `/system_messenger`

Node: `bag_recorder_node`

Node description Package: `bag_recorder` Alias: `N/A` Configuration Parameters:

IP address

Publishers: `/system_messenger` Subscriptions: `/system_messenger`, `/ain`, `/do`, `/din`, `/rtd`, `/bag_control`

Node: `labjack_ain_reader`

Reads up to 8 AI channels simultaneously from a single Labjack T8. This node uses a configurable timer to fetch (poll) channel reads from the T8. Package: `labjack_t8_ros2` Alias: `LabjackAINSlow` Configuration Parameters:

IP address

sample rate

topic name

Publishers: `/system_messenger`, `/ain_slow` Subscriptions: `none`

Node: `labjack_ain_streamer`

Streams up to 8 AI channels from a single Labjack T8. This node sets the T8 to stream data continuously at rates up to 40 kHz. Channel reads are simultaneous. To better manage network resources for high speed streams, data transfers are batched by the `ScansPerRead` parameter. Package: `labjack_t8_ros2` Alias: `LabjackAINFast` Configuration Parameters:

IP address

sample rate

scans per read

topic name

Publishers: `/system_messenger`, `/ain_fast` Subscriptions: `none`

Node: `labjack_dac_writer`

Controls one or more analog outputs on the T8. Output voltage can be adjusted between 0-10 VDC or ± 10 v with the LJTick-DAC Package: labjack_t8_ros2 Alias: LabjackDAC Configuration Parameters:

IP address

channel selection

topic name

Publishers: /system_messenger Subscriptions: /LJ_Ctl_Pub

Node: labjack_do_node

Switches one or more digital output (logic level) channels on the T8 high or low. Package: labjack_t8_ros2 Alias: LabjackDO Configuration Parameters:

IP address

topic name

Publishers: /system_messenger Subscriptions: /do

Node: labjack_dio_reader

Reads the logic state of one or more DIO channels. Package: labjack_t8_ros2 Alias: LabjackDIN Configuration Parameters:

IP address

topic name

Publishers: /system_messenger, /din Subscriptions: none

Node: adnav_driver

Manages configuration and initialization of the Advanced Navigation GNSS Compass and publishes its data. Package: adnav_gnss_compass Alias: AdnavCompass Configuration Parameters: too many to list here Publishers: /system_messenger, /nav Subscriptions: none

Node: xsens_mti_node

Manages configuration and initialization of the Xsens MTi-G-710 and publishes its data. This is a 3rd party package cloned from here. This can work with other Xsens inertial sensors too. Package: bluespace_ai_xsens_ros_mti_driver Alias: XsensINS Configuration Parameters: too many to list here Publishers: /system_messenger, /imu/acceleration Subscriptions: none

Node: ed582_driver

Reads up to four channels of RTD temperature measurements from a Brainbox ED-582. This is adapted from a 3rd party API available here. Package: ed582 Alias: RTDed582 Configuration Parameters:

IP address

sample rate

topic name

Publishers: /system_messenger, /rtd Subscriptions: none

Node: m2_control

This node can read directives published by the HMI and/or incorporate control logic and publish actions for digital or analog output channels. Package: m2_control Alias: M2Control Configuration Parameters:

IP address

sample rate

topic name

Publishers: /system_messenger, /dac, /do Subscriptions: /hmi_ctl

Node: rosbridge_websocket

This node creates a bridge between ROS2 and a websocket that is used to subscribe to and publish topics from a webpage using roslibjs. This is a lightly modified version of rosbridge_server from the rosbridge_suite github repo. Package: rosbridge_server_m2 Alias: Rosbridge Configuration Parameters: none Publishers: /hmi_ctl Subscriptions: /din, /ain_slow, /rtd

Real-Time

A real-time system is one that assures certain deadlines are met reliably within expected time constraints. See this Wikipedia article for a more complete definition.

!!! tip “Recommended Reading”

[This](https://shuhaowu.com/blog/2022/01-linux-rt-appdev-part1.html) series of

Using a real-time architecture and approach in MODAQ allows better management of sources of latency and reduction of jitter. What this means in practical terms is that certain time-critical loops, processes, or functions can execute deterministically and that variation in timing of successive iterations is kept low.

Latency

Opening Task Manager in Windows or the equivalent in other operating systems (OS), there may be hundreds of processes listed that are all vying for a slice of the CPU’s time. These include things like device drivers, software updaters, user software, malware detectors, and lots of other things that comprise the modern computing experience. With fast, multi-core processors, these appear to be seamlessly juggled, usually with little delay apparent to the user. This is all managed by the scheduler and it tries to be fair as it doles out access to a CPU core. Under critical analysis, an application may have to wait a non-deterministic amount of time for that CPU attention- and that wait time might vary considerably for each request. This variation may be only a few milliseconds but for DAQs running loops hundreds or several thousand times per second, that’s an eternity.

A properly configured real-time stack offers the software developer some tools to better manage how the real-time code gets its CPU time by preempting lower priority processes (jumping to the head of the line). The end effect is that the real-time code should experience less of this source of latency.

!!! info

Determinism

<p>

A deterministic real-time system should complete a task within a bound time period in a consistent and

Different real-time approaches yield better or worse maximum latency periods. Specific project requirements or use-cases often dictate the maximum acceptable latency for a particular application. This is most common for safety-critical systems, such as an anti-lock brake controller, where missing a deadline can result in disaster. These will often be associated with terms such as “guaranteed” or “hard” real-time. From a system architecture and software development standpoint, achieving this level of latency and determinism is difficult and probably not practical nor even necessary for a DAQ such as M2.

However, in the space between extremely low latency, safety-critical, real-time systems and general purpose operating systems without real-time support, there is Real-Time Linux. Or, more correctly, the PRE-EMPT_RT patch for Linux kernels.

Real-Time Linux

Linux with the PREEMPT_RT patch has been successfully used in various industries such as automotive, space, and industrial automation for years and by organizations including National Instruments, NASA, and SpaceX.⁶⁷ This lends confidence that it's appropriate for M2 applications.

At the time of writing, it appears that the mainline Linux kernel is expected to gain real-time support⁸ in a coming release. Until that happens, it's necessary to manually compile the kernel with the PREEMPT_RT patch or find a distribution for an already patched RT kernel.

Validating RT Performance

Once a patched kernel has been installed and the necessary changes to BIOS have been made, it's necessary to conduct some tests on the system to validate the 'as-built' latency performance.

A very useful tool for this is provided by intel: RTPM. This tool will evaluate the settings on your computer to ensure they are optimized for real-time performance. It also will conduct several tests that will characterize your real-time performance. For Arm64, consider using available benchmarking tools such as cyclictst.

Precision Time Protocol

The promise of PTP is that all connected PTP-aware devices will be synchronized and reference-time accurate to 100's, if not 10's of nanoseconds- without drift. By comparison, Network Time Protocol (NTP), which is what most consumer computers and laptops use as a time reference, can at best achieve 10's of milliseconds accuracy. While NTP accuracy might be perfectly acceptable to some and often better than a real time clock (which can drift several seconds a day), it's a ~6 order of magnitude difference in accuracy as compared to PTP. If a quality time reference is not available, samples taken from input modules on the same DAQ controller will be disciplined by the same clock and will likely be precisely timestamped relative each other, but may not be accurate to actual time (i.e. UTC) .

Where the real power of PTP is realized is in synchronizing samples taken by disparate systems. These can be DAQs on the same subnet, sharing the same master clock or systems completely disconnected from each other and separated miles apart. As long as each system is disciplined to a healthy PTP reference clock, the samples will be synchronized to the aforementioned nanosecond time precision. This is a boon for distributed architectures and system design flexibility, allowing multi-controller synchronization without physical interconnections.

In our experience, PTP implementation either works magically or is a real bear. It does help to have some knowledge of networking and configuring managed switches. PTP requires the following for optimal performance:

1. PTP time server
2. PTP-capable endpoints (e.g. M2 controller, ethernet based devices)
3. PTP services configured and running on the endpoints
4. PTP-capable network switch

These items need to be properly configured, otherwise the PTP performance may be degraded or non-existent. For instance, we've found that some cheap consumer unmanaged switches (such as the Netgear GS305) will pass PTP packets, however the observed PTP performance degrades to microsecond accuracy with large standard deviations. This is still pretty good and better than NTP, but not ideal and possibly not reliable.

⁶Using the minified version of any of the required JS libraries is totally optional. We prefer minified, since it's a smaller overall payload and can improve performance. The disadvantage to using minified libraries is that all unnecessary whitespace characters are removed, as are comments, and some optimizations may be applied (i.e. variable name shortening), so inspecting or editing the library code can be very difficult- even though it's still in plain text.

⁷We're using chartjs v3.3.2 since at the time we were creating the HMI there were compatibility issues with the plugin-streaming library on v4 and higher. That may have been resolved in later releases of plugin-streaming.

⁸Since finding the dependencies as minified libraries in the correct version can be a bit of a pain, here are direct links: luxon v3.4.40 and chartjs-adaptor-luxon v1.0.0

The linux support for PTP can be installed using `sudo apt install linuxptp`. This enables the `ptp4l` tool which allows user interaction with the PTP hardware clocks on your controller's NIC.

Command line startup of `ptp4l` (Intel controller):

```
=== "Intel" zsh      sudo ptp4l -m -s -H -i enp2s0 #replace enp2s0 with the NIC of choice
on your controller hardware=== "RPi CM5" zsh      sudo ptp4l -m -s -H --ptp_minor_version
0 -i eth0
```

`ptp4l` will run and display an output similar to this (use `ctrl-c` to quit):

```
modaq@m2-controller:~$ sudo ptp4l -m -s -H -i enp2s0
ptp4l[1576.346]: rms 228 max 382 freq -10480 +/- 259 delay 6276 +/- 37
ptp4l[1577.347]: rms 302 max 595 freq -10570 +/- 340 delay 6350 +/- 43
ptp4l[1578.347]: rms 165 max 286 freq -10514 +/- 191 delay 6364 +/- 25
ptp4l[1579.347]: rms 437 max 634 freq -10263 +/- 455 delay 6281 +/- 27
```

The `rms` field tells us our timing accuracy in nanoseconds for the samples. In this case we're achieving <500 ns accuracy. This value may be higher or lower in order of magnitude depending on the hardware used, but the goal is to achieve <10 μ s (10,000 ns) rms values and that they don't fluctuate wildly from sample to sample.

The role of `ptp4l` is to configure the PTP Hardware Clock (PHC) support and participate as a PTP client. However, to synchronize the PTP time to the system clock, an additional service is needed: `phc2sys`.

`phc2sys` is included in the `linuxptp` package installed earlier and can be run from the terminal as a test. For this to work, `ptp4l` must be running. Therefore, start `ptp4l` in one terminal window, open another terminal window and run `phc2sys`. In this example, the rms value of our system clock's accuracy is <500 ns.

```
modaq@m2-controller:~$ sudo phc2sys -s enp2s0 -w -m -u 2
phc2sys[1163.122]: CLOCK_REALTIME rms 384 max 386 freq -9958 +/- 55 delay 1684 +/- 2
phc2sys[1165.123]: CLOCK_REALTIME rms 95 max 126 freq -10228 +/- 67 delay 1683 +/- 10
phc2sys[1167.124]: CLOCK_REALTIME rms 323 max 456 freq -9950 +/- 220 delay 1656 +/- 36
phc2sys[1169.125]: CLOCK_REALTIME rms 464 max 554 freq -9514 +/- 18 delay 1690 +/- 6
```

Running `ptp4l` and `phc2sys` as System Services

In the previous example, `ptp4l` and `phc2sys` were running in the command line and while this works, they need to be manually started each time and occupy 2 terminal windows. Instead, we can make them into services that automatically run when the system boots up.

ptp4l To run as a service, there needs to be a valid `ptp4l.conf` file in `/etc/linuxptp/`. There should already be a file in that location, but it's probably configured wrong for this setup (such as turning on both server and client stuff). Suggest copying this file elsewhere for safekeeping then delete the contents and replace with (where `enp2s0` is replaced with your preferred NIC):

```
=== "Intel" zsh      [global]      time_stamping      hardware      clientOnly      1 # forces
client mode, prevents assuming server role      clock_servo      linreg      logging_level
4      summary_interval 5 # write to syslog every 2^5 seconds (32 seconds)      [enp2s0]
=== "RPi CM5" zsh      [global]      time_stamping      hardware      clientOnly      1
# forces client mode, prevents assuming server role      clock_servo      linreg      logging_level
4      summary_interval 5 # write to syslog every 2^5 seconds (32 seconds)      ptp_minor_version
0 # this entry is important for ptp to work on the CM5      [eth0]
```

`ptp4l` has many more settings available than the few used in these examples- and even these settings may have multiple options. Consult the `ptp4l` man pages for more info.

To start the service: `sudo systemctl start ptp4l`

To stop the service: `sudo systemctl stop ptp4l`

To enable the ptp4l service to start at boot: `sudo systemctl enable ptp4l`

To disable the ptp4l service starting at boot: `sudo systemctl disable ptp4l`

!!! warning “Troubleshooting” **Issue:** ptp4l will launch fine from command line, but will fail with syslog entries complaining about ioctl SIOCETHSTOOL failed, PTP device not specified..., and/or failed to create a clock when run as a service.

****Solution:**** (from: /usr/share/doc/linuxptp/README.debian):

1. Create a directory: /etc/systemd/system/ptp4l.service.d
2. In that directory, create a file with extension .conf (name does not really matter, call it ptp4l.conf)
3. Place the following lines in the file, replacing eth0 with desired port for instance enp2s0:

```
```zsh
[Service]
ExecStartPre=/bin/sleep 60 (NOTE: this is only needed if the start of ptp4l needs to be delayed- DO NOT REMOVE)
ExecStart=
ExecStart=/usr/sbin/ptp4l -f /etc/linuxptp/ptp4l.conf -i eth0
```
```

4. Reboot and service should be able to start as expected

****Issue:**** systemctl fails to launch with a Unit name ptp4l.service not found when trying to start ptp4l

****Solution:**** The ptp4l service file is misnamed.

Open a terminal in /lib/systemd/system and type: ``sudo cp ptp4l@.service ptp4l.service``

phc2sys To run as a service, first need to edit: /usr/lib/systemd/system/phc2sys.service.

NOTE: if phc2sys.service does not appear in the folder, but phc2sys@.service does, rename the file with this command in the terminal: `sudo cp phc2sys@.service phc2sys.service`

Open a terminal window in that folder and type: `sudo nano phc2sys.service` to open the file in a simple text editor.

=== “Intel” “zsh Edit the Execstart line to: Execstart=/usr/sbin/phc2sys -w -s enp2s0 -u 5

Save and exit nano.

enp2s0 should be the ethernet port connected to the subnet with the master clock

-u 5 writes to the syslog every 5 seconds, you can change this for more or less logging as desired.

```

=== “RPI CM5” “zsh Edit the phc2sys.service file with the following: (note: everything should be the same except for changes to the ExecStart= line)

```
[Unit]
Description=Synchronize system clock or PTP hardware clock (PHC)
Documentation=man:phc2sys
Requires=ptp4l.service
After=ptp4l.service
Before=time-sync.target
```

```
[Service]
Type=simple
ExecStart=/usr/sbin/phc2sys -w -s eth0 -c CLOCK_REALTIME -u 300
```

```
[Install]
WantedBy=multi-user.target
...
```

To start the service: `sudo systemctl start phc2sys`

To stop the service: `sudo systemctl stop phc2sys`

To enable the phc2sys service to start at boot: `sudo systemctl enable phc2sys`

To disable the phc2sys service starting at boot: `sudo systemctl disable phc2sys`

!!! warning “Troubleshooting” The instructions above edits the ptp4l.service and phc2sys.service in locations that could get overridden by a software update. It may be necessary to use a ‘drop-in’ .conf file to override desired settings. This file will live in `/etc/systemd/system/<serviceName>.d/override.conf`, where would be either ptp4l.service or phc2sys.service, depending on which you’re editing. The actual filename of the override file does not matter, as long as it lives in this folder and has the .conf extension. These steps may also be necessary if the services launch fine, but don’t appear to be using the settings you configured.

The contents of the ptp4l.service.d/override.conf would like like this:

```
=== "Intel"
 ``zsh
 [Service]
 ExecStart=
 ExecStart=/usr/sbin/ptp4l -f /etc/linuxptp/ptp4l.conf
 ...

=== "RPi CM5"
 ``zsh
 [Service]
 ExecStart=
 ExecStart=/usr/sbin/ptp4l -f /etc/linuxptp/ptp4l.conf -i eth0
 ...
```

And the phc2sys.service.d/override.conf:

```
=== "Intel"
 ``zsh
 [Service]
 Execstart=
 Execstart=/usr/sbin/phc2sys -w -s enp2s0 -u 5
 ...

=== "RPi CM5"
 ``zsh
 [Service]
 Execstart=
 Execstart=/usr/sbin/phc2sys -w -s eth0 -u 5
 ...
```

NOTE: the blank Execstart= serves to clear or reset the value placed by the original service file that

## Parsing MCAP Files

The mcap file format was designed by Foxglove and they also developed a GUI software that is able to process these data files and output plots, tables and other useful visualizers. For information on this GUI viewer, please see [Useful Links](#).

To analyze the data, we recommend using the rosbags python package which is also able to parse the mcap files and generate numpy arrays which can be used for data analysis. This can be installed with pip: `pip install rosbags`

Example Python code for parsing mcap files:

```
from rosbags.rosbag2 import Reader
from rosbags.serde import deserialize_cdr
import numpy as np
from matplotlib import pyplot as plt
from pathlib import Path
from rosbags.typesys import Stores, get_types_from_msg, get_timestore

import os

def get_all_file_names(folder_path):
 try:
 # List all files in the given folder
 file_names = os.listdir(folder_path)
 # Filter out directories, only keep files and return their full paths
 file_paths = [os.path.join(folder_path, file) for file in file_names if os.path.isfile(os.path.join(folder_path, file))]

 # Generate the modaq_messages/msg/{FILE_NAME} strings
 modaq_messages = [f"modaq_messages/msg/{os.path.splitext(file)[0]}" for file in file_names if os.path.isfile(os.path.join(folder_path, file))]
 return file_paths, modaq_messages
 except Exception as e:
 print(f"An error occurred: {e}")
 return [], []

Example usage
folder_path = r"C:\MODAQ\MODAQ2-RD-Dev\src\modaq_messages\msg"
file_names, message_types = get_all_file_names(folder_path)
print(file_names, message_types)
timestore = get_timestore(Stores.ROS2_HUMBLE)
add_types = {}

for i, name in enumerate(file_names):
 print("name: ", name)
 msgpath = Path(name)
 msgdef = msgpath.read_text(encoding='utf-8')
 print(msgdef)
 add_types.update(get_types_from_msg(msgdef, message_types[i]))

timestore.register(add_types)
create reader instance and open for reading
last_time = 0
last_timer = 0
dt_ros = []
dt = []

with Reader(r"./") as reader:
 # Topic and msgtype information is available on .connections list.
 for connection in reader.connections:
 print(connection.topic, connection.msgtype)

 # Iterate over messages.
```

```

for connection, timestamp, rawdata in reader.messages():
 if connection.topic == '/ain_flap':
 msg = typestore.deserialize_cdr(rawdata, connection.msgtype)
 time = (msg.header.stamp.sec + (msg.header.stamp.nanosec/1e9))

 dt_ros.append(time-last_time)
 last_time = time

plt.plot(np.array(dt_ros)[100::])
plt.show()

```

## ADCs

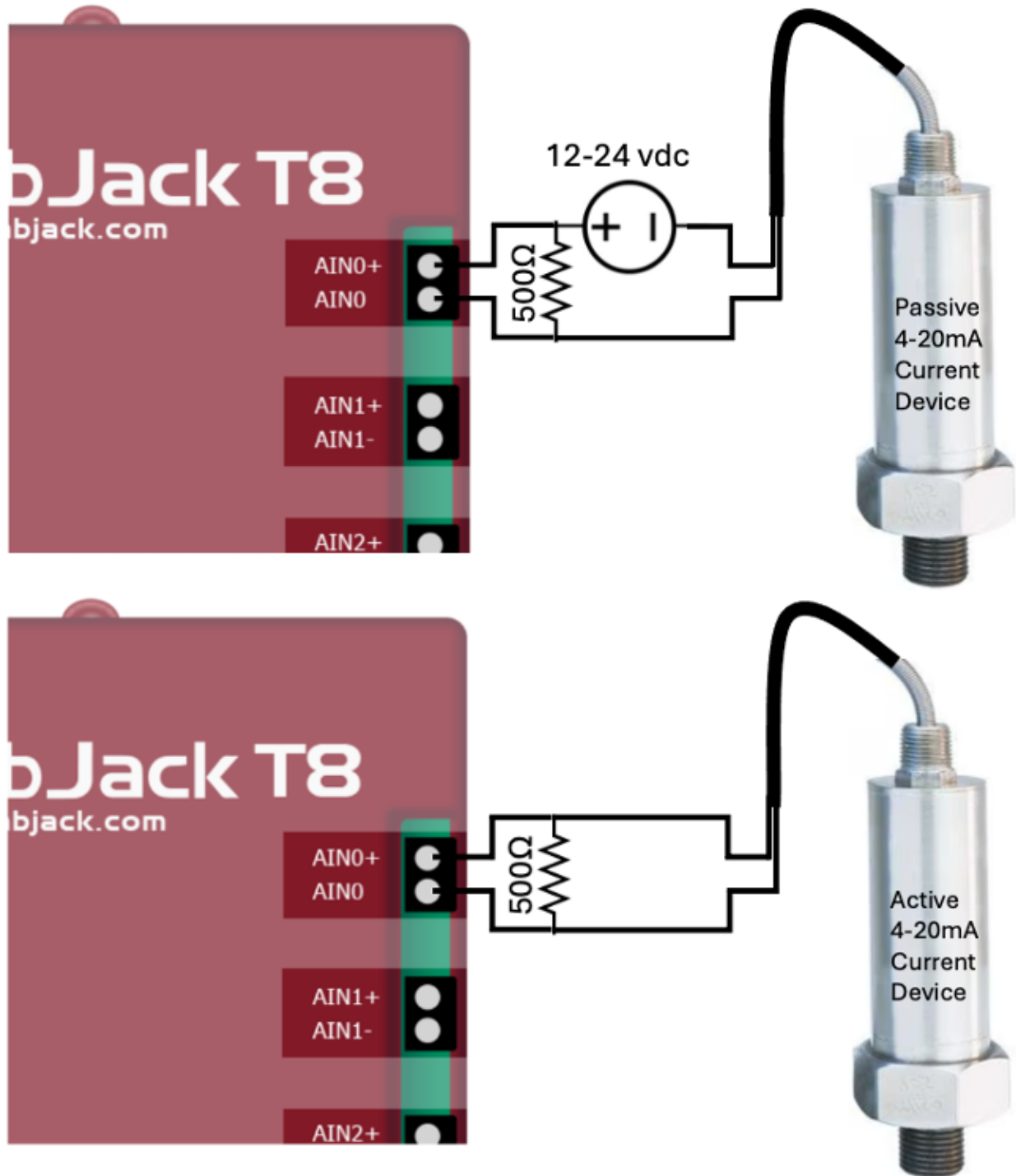
While there's an abundance of really good analog to digital converters (ADCs) on the market, one of our biggest challenges was finding one suitable for our requirements.

- 24 bit
- Simultaneous sampling
- At least 4 channels
- Channel-to-channel isolation
- Anti-aliasing filter
- Not dependent on a particular vendor's controller (i.e. non-proprietary)
- Ready to go, with simple ethernet or USB interfacing
- Works with linux

## 4-20 mA Current Loops

One or more of the 8 analog inputs available on the RD could be allocated for measuring outputs from devices that signal using a current loop. The 4-20 mA current loop would be converted to 1-5 volts using a  $250\ \Omega$  (or 2-10 VDC using a  $500\ \Omega$ ) precision shunt resistor connected across the positive and negative input terminals of the voltage channel.

Alternatively, M2 supports using current loop interface modules with digital outputs (example 1, example 2), which is not included in the RD specification, but available upon request.



It's important to note that there are 2 types of current loops: passive and active. Passive devices (such as some pressure transducers) require an external source driving the current loop, while active devices provide the driving source internally. The M2 RD supports active loops through the shunt resistor method mentioned in the first paragraph. Passive circuits can be made active with the addition of a power supply. The example 1 link for the optional current loop interface works with passive devices.

## Heading

Getting a reliable estimate of heading on a fixed or semi-stationary device can be tricky. Devices that rely on Earth's magnetic field need to be calibrated in-situ and have the proper magnetic declination correction applied.

Since a calibration procedure generally requires rotating the sensor  $360^\circ$  in one or more axis, this is often difficult or near impossible once the sensor is mounted on the device (WEC, buoy, etc). It's easier to calibrate the compass prior to installation, however this is not effective, since the magnetic field will most likely be distorted by the presence of hard and soft iron sources near the mounting location.

This leads to our decision to include the Advanced Navigation GNSS Compass in the M2 RD design specification. It avoids the issues associated with magnetic compasses by using dual GPS antennas to achieve  $0.2^\circ$  heading performance. Because of the antennas, it needs a clear view of the sky- which might not be possible in some use-cases. Therefore, the magnetic heading from the Xsens INS can be used instead.

If very precise heading estimates are required in your application and GPS is not an option, we suggest looking at some gyrocompass options, such as the ring laser or fiber optic gyros.

## M2 HMI MODAQ M2 HMI Example Design

A Human-Machine Interface, or HMI, is a control panel or portal that enables high-speed, near realtime interaction with M2. It can display or plot values and allow for operator control of specifically coded elements running in M2- such as manual control of a relay.

Typically, the HMI would be referred to as the *frontend*, while the bulk of the MODAQ code responsible for the actual data acquisition, control outputs, and general logic is considered the *backend*.

M2 uses a standard web development stack consisting of HTML, JavaScript (JS), and CSS to render the HMI. A web server runs on the M2 controller and users (clients) connect to the HMI using their web browser. For security purposes, the HMI is only accessible if the client is on the same subnet as the M2 controller. Remote access from anywhere in the world could be accomplished using a VPN gateway.

The frontend and backend communicates through the use of a websocket, which provides a dynamic, bidirectional interface to exchange data through ROS2 topics or services.

### Getting Started

This will not be a detailed tutorial and it assumes some familiarity with the ROS2 environment, Linux terminal, and web server/client basics. These are the following minimum requirements to get an M2 HMI up and running:

Backend:

- ROS2 Humble installed on the M2 controller. This requirement should already be met if ROS2 was installed in the previous section on getting MODAQ up and running.
- `rosbridge_suite`. This creates the backend service to allow frontend interaction with the ROS2 environment.
- nginx web server. Web servers such as `lighttpd`, the venerable Apache, or numerous others could be used instead of nginx. The choice of web server is mostly inconsequential as long as it supports modern web technologies and is reasonably lightweight. We will only provide guidance for configuration and use of nginx.
- At least one ROS2 node running on the M2 controller as publisher and optionally a subscriber node.

Frontend:

- An IDE or text editor for composing the web documents. If you're using VS Code for ROS2, that will work fine.
- `roslibjs`. This will be installed as part of the `rosbridge_suite` on the backend, but needs to be 'visible' to the web server. Visibility might involve providing the correct path to the library or our preference, place of copy of the minified (alternative link) `roslib` in the `/js` folder in your nginx html document root.<sup>9</sup>
- `chart.js` v3.3.2<sup>10</sup> (minified). This is only required if you want to have plots in your HMI.
- `chartjs-plugin-streaming` (minified). and its dependencies<sup>11</sup> (which are noted in its Getting Started section of the `chartjs-plugin-streaming` webpage linked in the last sentence). This is only required if you intend to have plots in your HMI and want those plots to update in near realtime (AKA streaming).

!!! note

The code uses offline bundled versions of each of these libraries, which allows the HMI to operate with

---

<sup>9</sup>Using the minified version of any of the required JS libraries is totally optional. We prefer minified, since it's a smaller overall payload and can improve performance. The disadvantage to using minified libraries is that all unnecessary whitespace characters are removed, as are comments, and some optimizations may be applied (i.e. variable name shortening), so inspecting or editing the library code can be very difficult- even though it's still in plain text.

<sup>10</sup>We're using `chart.js` v3.3.2 since at the time we were creating the HMI there were compatibility issues with the plugin-streaming library on v4 and higher. That may have been resolved in later releases of plugin-streaming.

<sup>11</sup>Since finding the dependencies as minified libraries in the correct version can be a bit of a pain, here are direct links: `luxon` v3.4.40 and `chartjs-adaptor-luxon` v1.0.0

This is admittedly a regression from modern web-stack architecture practices, however we're purposely p

## nginx Configuration

nginx can be install with the following terminal command:

```
sudo apt install nginx
```

Configuration of nginx will live in these folders:

```
/etc/nginx/sites-available
```

```
/etc/nginx/sites-enabled
```

The website (HMI) is hosted from the `/var/www/` folder by default. Multiple websites can be hosted simultaneously with nginx and they can be kept organized in their own folders. This discussion will assume a folder called `modaq_hmi` was created in `/var/www/`

Since the config and content folders live in root, permissions will be necessary to manipulate these files from a standard user account. Using temporary privilege escalation (`sudo`) will suffice for the nginx configuration files, but this can get tedious for the HMI content pages. Best Practices would be to edit these files in user space (i.e. a folder downstream of the user's `/home` folder) and include them in version control (github), then copy them to the production space (`/var/www/modaq_hmi`) when ready to take them live.

**Set User Permissions to nginx Web Page Folder** While this is optional, it will make life easier during the dev process to add user permissions to thw `/var/www/modaq_hmi` folder. Otherwise, every single change will require sudo privilege escalation.

This sets the local user 'modaq' with `rwX` privileges in the `modaq_hmi` folder recursively:

```
sudo setfacl -R -m u:modaq:rwX /var/www/modaq_hmi
```

It may be necessary to do a `chmod` afterwards:

```
sudo chmod -R 755 /var/www/modaq_hmi
```

**Create nginx Virtual Server Configuration File for HMI** As noted earlier, nginx creates two folders for the server configuration. The `sites-available` folder contains the configurations for one or more websites, while the `sites-enabled` folder contain the configuration(s) for the site(s) that nginx is currently serving. Think of `sites-available` as the parking area and `sites-enabled` as the active/production area. Instead of actually moving or copying the configuration file to the `sites-enabled` folder, nginx expects only a symbolic link to the file in the `sites-available` folder. The following shows the contents of a configuration file that works for us:

```
server {
 listen 80;
 listen [::]:80;

 root /var/www;

 # Add index.php to the list if you are using PHP
 index index.html index.htm;

 # For this to work, needs DNS resolution or entry in
 # hosts files on client computer
 server_name modaq.hmi www.modaq.hmi;

 location / {
 # First attempt to serve request as file, then
```

```

 # as directory, then fall back to displaying a 404.
 try_files $uri $uri/ =404;
 }
}

```

A couple of notes about this configuration:

1. This configuration is for an unsecured connection (i.e. not https://) and is probably safe for the private subnet which serves this HMI. However, the commented section on **SSL configuration** could be enabled to provide secure connections- if certificate dependencies are met. Some web browsers might complain about connecting to the HMI citing that the connection is unsecure and potentially dangerous if SSL is not enabled. It may be necessary to click through some warnings or whitelist the site.
2. The entry for **server\_name** was an attempt allow users to type modaq.hmi into their browser instead of having to know the IP address of the controller. While that would be convenient, more work needs to be done to get it working, so we backburnered this for other priorities.

**Ubuntu Firewall Rules** If the Ubuntu firewall (ufw) is enabled on the controller, it may be necessary to create a firewall rule to allow nginx to access the network. This is done with a simple terminal command:

```
sudo ufw allow 'Nginx HTTP'
```

## ROSBridge Server

The primary mechanism providing connectivity between ROS2 nodes running on M2 and the nginx web server is a websocket that is managed through rosbridge and roslibjs. Rosbridge is the service running along with M2 ROS2 nodes and exposes all published topics to a websocket.

If the rosbridge\_suite was properly installed, the rosbridge server can be started with this command:

```
ros2 run rosbridge_server rosbridge_websocket
```

It's recommended to add this as the last item in the M2 launch file so that it starts automatically with the rest of the ROS2 nodes.

```

Node(
 package='rosbridge_server',
 executable='rosbridge_websocket',
 name='rosbridge'
)

```

## Web Client Framework

There are four main parts of the web code that are necessary for pub/sub with ROS2 on the M2 system

1. Establish the websocket connection to ROSBridge
2. Create a listener for ROS2 topics (subscribe) and/or define a topic to publish
3. Handle incoming/outgoing messages
4. Create the user interface (actual webpage)

## Basic HMI Design Elements

The M2 HMI uses a simple but effective grid of tiles to organize the content. This is accomplished through a hierarchy of <div> tags to define the sections (or divisions) with attributes applied from CSS classes.

In this public release, these basic features are included in the HMI:

1. Traffic Lights - These are multi-modal text boxes that can change color based on logic contained in the accompanying JS. The traffic lights can be used to indicate whether a parameter is nearing or exceeding defined thresholds or indicate an alarm condition.

2. Scalar Data - Display the current numeric value of certain parameters from a subscribed topic.
3. Streaming Plots - The examples include single and multivariate plots, as well as dual y-axis plots.
4. Boolean Tick Boxes - These allow publishing boolean values back to M2 that can be used for manual control or setting the state of certain features.
5. Text/Numeric Entry Boxes - These can publish numeric or text values to M2.

Since everything is written in ordinary HTML/JS/CSS and a handful of publicly available JS libraries, users can easily leverage these examples to develop a customized HMI suitable for their application.

The tile system (aka grid) is quite flexible, allowing one to many content tiles on each row. While the height of each row can be constrained, the example included in this repo allows the height to grow to fill the content. The width of each tile is constrained to a fraction of the overall width allowed by the top-level (parent) div container.

Diving deeper, the parent div is configured with 4 available columns in the included `style.css` file and looking at the screen shot of the HMI page at the beginning of this section, the MODAQ banner is allowed to span 3 columns in width, while the rosbridge status tile is set to 1 column- for a grand total of 4 columns.

**roslib Subscriber** roslib.js is a javascript library used on the webserver that can subscribe to topics published on M2 and display the data in a webpage. In addition it allows for publication of topics from the webpage as well as using or creating services.

First, an object is created to establish the websocket connection to the ROS2 instance on the M2 controller (showing the most relevant code snippet):

```
const ros = new ROSLIB.Ros({ url: "ws://localhost:9090" });12
```

Next, a listener object is created to subscribe to the desired topic being published on the controller. In this example we have a temperature and humidity sensor that is acquired in M2 using MODBUS TCP and the scaled data are published to the `ths` topic.

```
// MODBUS Temperature and Humidity Sensor
var ths_listener = new ROSLIB.Topic({
 ros: ros,
 name: '/ths',
 messageType: 'ths_modbus/msg/Th',
 throttle_rate: 1000 // throttle_rate sets update interval in ms
});
```

When a new message is received, it's copied to a variable with global scope for further manipulation within the curly braces.

```
ths_listener.subscribe(function (message) {
 // copy ROS data to var with global scope
 ths_data = message;
 // do stuff here with the topic data
});
```

**roslib Publisher** The roslib publisher follows a similar pattern to the Subscriber, first a topic object is created:

```
var hmiCtl = new ROSLIB.Topic({
 ros: ros,
 name: '/hmi_ctl',
 messageType: 'modaq_messages/msg/Hmi'
});
```

---

<sup>12</sup>Replace 'localhost' with the IP address of the M2 controller hosting the HMI webpage.

We had previously created a unique message schema for the HMI in ROS2 on the M2 controller and included it during the build process, which includes the parameters we want to publish from the HMI.

An event listener is bound to the “Push to MODAQ” button in the HMI, triggering a read of the user controls...

```
var hmiPub = new ROSLIB.Message({
 override: user_override,
 do0: user_do0,
 do1: user_do1,
 do2: user_do2,
 do3: user_do3,
 int1: user_int1,
 message1: 'Hello Casey',
 message2: 'Hello Aidan',
 float2: 3.17,
 header: {
 frame_id: '1'
 }
})
```

and publishes them.

```
hmiCtl.publish(hmiPub);
```

**Traffic Lights** The appearance of the traffic lights is done with CSS and the color change is triggered by some JS logic.

```
if (ths_data.humidity > 90) {
 document.getElementById("status1").innerHTML = "Humidity"
} else if (ths_data.humidity > 65) {
 document.getElementById("status1").innerHTML = "Humidity"
} else {
 document.getElementById("status1").innerHTML = "Humidity"
}
```

In this snippet, if the humidity is >90, the traffic light identified by the tag `status1` is styled using a combination of two CSS classes: `traffic_light false`, which in this case will color it red. Similarly, if it's >65, it's colored yellow, otherwise it's green.

**Streaming Charts** We chose chart.js as the charting engine for the M2 HMI, since it's highly customizable, capable, and looks good. It also has great support (both from the developer and user community), which is important, since it can be a little complicated to use. Because of this complexity, this section will not go into detail on its use, but will point out a few details around getting the streaming to work correctly - particularly with multivariate/dual-axis plots.

chart.js, without the `chartjs-plugin-streaming` plugin, expects to be passed a static numeric array and it renders into a plot with the selected attributes. The streaming plugin (basically) creates a fifo buffer for the data points and animates them on the x-axis.

!!! tip

limit the chart history in order to assure smooth operation- especially with multiple charts on a single

The architecture of the HMI web stack purposely places the computational burden of chart composition and rendering on the web client (i.e. your browser) not the M2 controller. Therefore, low-powered computers may struggle with smooth animation if the HMI has a lot of plots and/or the chart history is too big. The example included in this repo updates the chart at 1 Hz and displays up to 1 minute of data history. The update rate and history depth can be adjusted to preference.

chart.js breaks up the chart composition into 3 blocks: setup, config, and render. The setup block defines the y-axis datasets, labels, and plot trace attributes. The config block expands on this by defining additional attributes and binding the dataset(s) to an object that gets rendered in the render block. This all expects the data to be formatted as an array, however our data are constructed into objects.

To overcome this, we define a object literal to map the y-axis label(s) to the label used to identify the desired element in the data object (example for the accelerometer data).

```
const dataMap = { Ax: 'ax', Ay: 'ay', Az: 'az' };
```

Then we insert each data point into the chart object:

```
chart.data.datasets.forEach(dataset => {
 dataset.data.push({
 x: Date.now(),
 y: mpu6050_data[dataMap[dataset.label]],
 });
});
```

Side Note - We typically use high quality inertial sensors in our projects and eagle-eyed readers will notice the MPU-6050 referenced in a few places in this documentation and the github repo. The MPU-6050 is a <\$5 sensor that has some utility in the occasional low-cost/low demand applications (see: MODAQ-BB).

Similarly, handling a multivariate data object and plotting with dual y-axes took some creativity, but is simpler as long as only 2 parameters are plotted:

```
chart.data.datasets.forEach(dataset => {
 dataset.data.push({
 x: Date.now(),
 y: dataset.label === 'Temperature' ? ths_data.temperature : ths_data.humidity
 });
});
```

# M2 Configurator MODAQ M2 Configurator Guide

Configurator MODAQ2 Repo: MODAQ2/configurator

Several parameters within M2 can be configured through the use of a yaml file that is located in `/src/m2_core/m2_launch/config`. This allows the M2 source code to be somewhat stateless, while allowing common or project-specific changes to be applied at runtime.

While the yaml file is plain text and can be edited in any basic text editor, this is not recommended since invalid inputs or formatting errors could result in unintended settings or render the file unusable. The Configurator exposes all available configurable items for the selected M2 nodes and performs validation on user inputs.

## Features

- Edit an existing configuration or create a new one from scratch
- Auto archiving of loaded config files
- Message box at bottom of window to display context-aware help or error info
- Input validation to assure values typed by user are acceptable
- Light/dark mode

Figure 1. M2 Configurator screencap. Mouse over to reveal legend.

if you mouse over the image above, reference numbers appear next to the GUI elements that correspond to the following list of explanations:

1. Load Config button: This will open a file browser window for loading an existing configuration. A copy of the loaded file will automatically get created as a backup. This is unnecessary when creating a new configuration.
2. Write to File button: When done editing a configuration, select this button to save the configuration to a yaml file.
3. Switch between light and dark modes.
4. Node selection pane. A node is an M2 software module containing configurable items that is launched as part of functional M2 system. (see: [ROS2 Nodes](#))
5. Selection dropdown for available nodes that can be added to the configuration.
6. Window showing nodes that have been added to the configuration. If an existing configuration was loaded, the nodes in that file will populate here. On a new configuration, this window will be empty until nodes are added from the dropdown box (5).
7. Rename the selected node. Please see section XXX for node naming guidance.
8. Delete the selected node. Use with caution: there is no undo for this.
9. Configuration pane
10. Window for viewing and editing configuration values.
11. Double clicking on a value will enable editing of that value. Some items, such as Channel Names may have more than one value. In these instance, double clicking on the text will expand (or unhide) the values.
12. Example of the expanded view for parameters with multiple values. These can be edited or deleted individually. New values can be inserted (up to limits specified in the schema).
13. Delete Selection button: this will delete the highlighted parameter and value. There is no undo.
14. Insert Here button: On Parameters with multiple values, a new value can be inserted at the highlighted location. NOTE: new parameters cannot be added, only new values for existing multi-value parameters.
15. Message Box: Helpful test will appear here as the user clicks on elements in the Configuration pane.

## Getting Started

The M2 Configurator is written in Python and uses a couple of popular libraries. Our original intent was to offer standalone executable versions of this code so that it would not be necessary for end users to have or install python on their systems. However, executables may be blocked by policy on managed computer

systems or flagged as potential malware due to the lack of a signed certificate. Since, it's assumed the likely users of the M2 Reference Design and this configurator have some level of skill with programming, we're distributing just the source code and the following setup guide. Users can compile executables if so desired using utilities such as Pyinstaller.

## Recommended Setup

While not absolutely necessary, we do recommend creating a Python virtual environment to edit and run the Configurator. The virtual environment or venv encapsulates python and the required libraries in a separate 'workspace' and has certain benefits. If you prefer not to use a venv, skip steps 2 and 3 below.

1. In a terminal, navigate to or create a folder for your M2 Configurator workspace.
2. Create a new venv for the M2 Configurator workspace: `python3 -m venv modaq_venv` (replace `modaq_venv` with your preferred name)
3. Activate the workspace: MacOS/Linux: `source modaq_venv/bin/activate` or `source ./modaq_venv/bin/activate`  
Windows (cmd): `modaq_venv\Scripts\activate` or `modaq_venv\Scripts\activate.bat` Windows (PowerShell): `modaq_venv\Scripts\Activate.ps1`
4. Install required libraries.
  - `pip install -r requirements.txt` or if you prefer not to use our requirements.txt file
  - `pip install PyYAML customtkinter CTKMessageBox CTKListbox`
  - `pip freeze > requirements.txt` (optional)
5. Place the M2 Configurator python script (`m2conf.py`) and schema (`m2_config_schema.yaml`) in the workspace folder.
6. Done!

## How to Use

Assure that the venv has been activated (if using a virtual environment) then launch the Configurator: e.g. `python m2conf.py`

!!! Note The numbers shown in parentheses below correspond to the numbers revealed when mousing over Figure 1.

An existing M2 yaml configuration file can be loaded for editing or a new one created from scratch. Much of the Configurator operation is the same either way.

**Load an Existing Configuration File** Select **Load Config** button (1) and select an existing yaml file from file dialog box that pops up. The M2 nodes available in that configuration file will be populated to the node selection listbox (6).

**Starting from Scratch** Select the **Add Node** dropdown box (5) and select a node to add to the configuration in the node selection listbox (6). Repeat this process until all nodes are added to listbox.

!!! Warning

Do not add two or more of the same node type without first renaming the node after it it added. For ins

**Viewing and Editing the Configuration** Selecting any node that appears in the listbox (6) so that it is highlighted will result in its associated Parameters and Values (key/value pairs) being displayed in the Configuration Values (10) window.

Parameters can be highlighted by mouse selection and help or info text for that parameter will appear in the Message Box (15). A highlighted parameter and its value(s) can be deleted with the **Delete Selection** button. However there is no **undo** for this and inserting new parameters is currently not possible within the Configurator.

The Values side of the Configuration Values window is editable. Double clicking on any value will allow the cell to be modified. Pay attention to the help text in the Message Box (15) for the type and/or format of input

expected (e.g. integer, float, string, IP address, etc.). The configurator does perform some input validation (watch the Message Box for feedback), however some improper inputs may be accepted- particularly string type inputs.

Some Parameters/Keys expect allow for more than one value. These values are displayed collapsed with <double click to expand> to show the list. Editing, deleting, and inserting are allowed in these lists. Some parameters have limits on the number of values that can be added to a list. For instance an 8 channel analog input node cannot have more than 8 channel names configured. The Configurator will warn the user if they attempt to add more values than the parameter supports in the Message Box.

??? Info “Q&A” **Q: Why can’t I add a new parameter to a node?** A: The C++ code for a node must be specifically written to accept configurable values at launch. All such configurable values for a node are already presented when a node is added to the configuration.

**Q: What happen with M2 if I deleted a parameter, will it still work?** A: Depending on which parameter was deleted there may be no noticeable impact to the operation of M2. This is because all configurable parameters are hard-coded with default values that are used if values are not passed in at launch. However, those default values may cause undesirable behavior if a parameter is missing such as with email addresses or file paths.

**Q: How do I get a parameter back if I accidentally delete it?** A: At this time, there is no direct way to insert a parameter- maybe in a future release. However as a workaround, delete the entire node from the node listbox and add the node back again. Any previous edits on the deleted node will be lost, but the deleted parameter will now be available.

**Q: I renamed a node and it does not work when I launch the M2 code** A: Node names in the configurator must match their launch names in the M2 code see: [Link](#).

**Saving a Configuration** The edited configuration can be saved to a file by selecting the **Write to File** button. A file dialog will appear with either the loaded filename prepopulated (in the case of a loaded config) or a default name (in the case of a new configuration). The user can override this and name the file something else- however this filename must match the config filename specified in the M2 launch file (see: [Link](#)).

## Technical Details

The M2 Configurator is written entirely in Python 3.12 (specifically: 3.12.7) and is using Tkinter from the standard library for creating the GUI with added visual flare from CustomTkinter (along with the widgets CTKListbox and CTKMessagebox).

**The Schema** On launch, the code loads the file `m2_config_schema.yaml` into a dictionary object `schema_in` and this dictionary is referenced during runtime for a number of purposes. This allows the general Configurator code to be mostly stateless and as new nodes or configurable parameters are added to M2, only the schema needs to be updated to enable these new features.

The schema contains runtime settings for all currently released M2 nodes, as well as associated key/value pairs that the Configurator uses to validate user input and display help information. These special configurator-only keys contain a suffix with an underscore `_` followed by a token, such as `_dt` (for datatype). The following are examples of schema-specific subkeys and how they are used in the Configurator:

`channelScales_dt: float` - The expected datatype of the user input `channelScales_max: 8` - Maximum number of entries allow for this key `channelScales_help: "Enter the scaling value for the channel as a floating point number"` - Text to display in the message box

In cases where the user renames a node, the Configurator will insert a key `schema_template` into the working dictionary that references back to the original key name in the schema so that input validation and help text continue to work.

**Working Dictionary** A second dictionary is created by the configurator which is used to store a ‘working’ copy of the configuration the user is building and editing. If an existing yaml file is loaded into the Configurator, the entire contents of that file are written to the working dictionary. In cases where the user starts from scratch or adds new nodes from the Nodes List dropdown menu, the selected nodes are written to the working dictionary. The contents of the working dictionary are used to populate the nodes that appear in the Nodes List listbox and the key/value pairs that appear in the Configuration Values table.

When the user selects **Write to file**, the contents of the working dictionary are converted to yaml and written to the output file. The working dictionary can be viewed at any time in the terminal window by clicking the **Preview** button.

!!! Note The **Preview** button was used for debugging during the development process but was considered to retain value for general users of this code. It should be disabled if the code is compiled into an executable, since a terminal window will be unavailable. A future version of this code may print the preview to a spawned text window instead of the terminal.

**The Classes** The code is organized as a collection of classes that encapsulate the various GUI and logic elements. In total there are four classes that do the following:

**App** Top-level class that defines the overall GUI window parameters, including the grid layout of the GUI elements. It also includes the methods for loading and saving the yaml files. **NodesFrame** Defines the look and functionality of the Nodes List pane on the left side of the GUI window. It contains methods for managing the included listbox contents. **ValuesFrame** Defines the look and functionality of the Configuration Values pane and message box on the right side of the GUI window. It contains methods for displaying and manipulating parameter values and updating content the message box. **EntryPopup** This class contains the logic to enable the (apparent) in-place editing of the values in the Value column of the Configuration Values pane. The Configuration Values pane uses the Treeview widget from Tkinter to create the table view. Unfortunately, Treeview does not contain a native method for directly editing table values by a user in the GUI. EntryPopup gives the illusion of directly editing the table values by creating a small popup modal sized and placed over the field the user double-clicked to edit. After the user enters their desired value and hits enter/return, that value is updated to the table underneath the popup and the popup is then destroyed. From the user’s perspective, it appears they directly edited the table.

**Assuring Compatibility with the M2 Code** Some of the actions allowed by the Configurator can break compatibility with the M2 software. For example, while renaming nodes is allowed, the new name must match the node name assigned to the node in the M2 launch file (`m2_launch.py`) settings- otherwise the renamed node will be ignored when M2 launches. A node can be given any name in the launch file as long as it meets syntax requirements and is unique. Therefore, when it comes to renaming nodes, it may be necessary to also change the name (manually) in the launch file too.

Similarly, the file name given to the configuration file must agree with the filename specified in the launch file.

The C++ language that M2 is written in is a strictly-typed language and in such, if a node expects an integer datatype for a configurable value, yet a float datatype is provided from the configuration file, that node will issue an error and fail to launch. For this reason, the Configurator schema includes the expected datatype for each parameter and validates user input to reduce the likelihood of this type of error occurring. In some cases, the Configurator will attempt to coerce the input to the correct datatype, but sometimes this is not possible (see **Known Issues**). An input of 20 is an integer, while 20. (with a decimal point) is a float.

## Known Issues

There are some known issues present in this release of the M2 Configurator. These are expected to be addressed at a later time, but are considered either ‘Quality of Life’ or edge-case issues- so not critical.

- Values window (Treeview) does not properly respond to dark mode

- If a user inputs a float where an int is expected, operation will not complete and displays an error in the message box. Expected behavior is to coerce float to an int- however this ‘bug’ might be considered a feature since it requires the user to fix the erroneous input.
- If user renames a node to a name that already exists in the listbox, weirdness ensues. Need to add a check to the validation method to prohibit duplicate names.
- Similarly, if a user adds two (or more) nodes of the same node to the listbox from the dropdown menu, they will not be unique- rather they point to the same object. Workaround is to add the first node and rename it, then add the second node. This is more commonly a problem with larger M2 designs that have multiple input modules of the same type, such as 2 LabJacks performing analog input operations.
- If an existing config is loaded that contains nodes named differently than the names used in the schema and does not contain a valid `schema__template` key, any edits in the Configuration Values window (ValuesFrame) will throw an error.
- It’s possible on some systems that the executable version of the configurator may be disallowed either due to policy (managed computer and user does not have sufficient permissions) or malware detection (the app is distributed without a signed certificate). For these reasons, a standalone executable of the configurator app will not be offered on this repo.

## Acknowledgements

The in-place editing technique used in this app, which allow for double-clicking on an item to change its value in the Configuration pane is based on a tutorial published on [CodersLegacy.com](https://coderslegacy.com).

## Q&A MODAQ M2 Q&A

### How does MODAQ differ from pre-built commercial options?

This is a good question and important to understand where MODAQ fits in with commercial alternatives. Due to a variety of factors, the answer is not that simple. One needs to consider budget, performance expectations, system flexibility, functional requirements, and the technical aptitude of the end user.

Here's some things to consider:

1. Data acquisition systems can be found ranging from a few hundred dollars to well over \$100,000 in cost. The price point is often a clue as to how performant and capable (and complex) the system will likely be.
2. While some commercial products provide user software to configure, manage, and operate the system, anything beyond the basics will require programming.
3. If the measurements are important, for instance being used for model validation, design basis, safety, or for financial/production purposes, the quality and integrity of those measurements must be able to withstand scrutiny.

Data acquisition is an engineering discipline with numerous sub-specialties that can take years and lots of study to become competent. By offering MODAQ publicly, users can leverage the efforts of the national lab

### Why not just use generic DAQ controllers?

There are a lot of options out there, but it does not take long to realize that:

1. You're just trading one ecosystem for another.
2. Specification to price ratio might be (much) worse.
3. Documentation and libraries may be lacking.
4. Device compatibility may become a bit of an issue.
5. Your preferred programming language or operating system might not be supported.
6. Software still needs to be developed (or in some cases configured) to achieve expected results.
7. It could end up being a lot of work!

Of course it depends on individual needs and expectations, but whether going with MODAQ or an alternative, there's no getting away from the tasks of programming the system and assuring it will perform to expectations and deliver quality results.

Some vendors sell controllers and I/O modules and offer software packages that require some configuration to achieve basic and limited data acquisition that can be considered on the easy end of the spectrum. However, doing anything beyond simple acquisitions often requires accessing a vendor supplied API and writing code in a traditional language like C/C++ or python - or worse - writing code in a vendor developed scripting language.

There's a class of controllers that include an array of built-in I/O that are nice, well thought out, easy to configure, and performant, however under closer inspection, they lack some key features that make them unsuitable in an embedded application deployed in the field. For example, these options generally have limited data logging capability, supporting only small (usually 64 GB or less) and slow microSD cards. There's also limitations when it comes to more technical matters, such as sampling speed, multiplexing (or scanning) vs simultaneous sampling, data synchronization, bit depth, multitasking, determinism, and noise. They also lack flexibility of local filtering or signal processing of data, local QC, and communications/remote supervision. Of course, use case and measurement objectives matter, so these can be perfectly fine for some applications.

### Is MODAQ plug-n-play?

Generally, no. Each user will have their own unique set of measurement objectives and requirements which cannot realistically be fulfilled by a general MODAQ design. The one exception is the Reference Design

presented in the previous sections of this documentation. If the user selects the exact hardware presented in the [Hardware Reference Design section](#) and follows the instructions in the [Software section](#), the system should function. NLR has a limited number of fully built and configured MODAQ2 Reference Design systems that can be loaned out under the M2GO program.

## **What is M2GO?**

M2GO is a fully built, configured, functional, and tested manifestation of the M2 reference design presented in this web document that is available for loan from NLR to qualified 3rd party end users. This is first come, first served and availability is limited. To learn more, contact us [here](#)

## **Do I need to know programming to use MODAQ?**

## **Which sensors or instruments should I use?**

If you have to ask...

## **Can NLR help configure or customize MODAQ?**

MODAQ is funded by the Department of Energy's Hydropower and Hydrokinetic Office (H2O) for the purpose of advancing the development and testing of Marine Energy devices. There are mechanisms in place for large and small organizations to engage NLR. Contact the development team.

## M2 Useful Links MODAQ M2 Links

!!! note When exploring learning material for ROS2, be wary to avoid training on ROS1 as it is quite different from ROS2. Try to stick to training specific toward ROS2 Humble Hawksbill.

### MODAQ

M2 Docs Repo: [github.com/NatLabRockies/MODAQ2](https://github.com/NatLabRockies/MODAQ2)

NLR: [nlr.gov/water/open-water-testing.html](https://nlr.gov/water/open-water-testing.html)

### ROS2

Design Reference: [design.ros2.org](https://design.ros2.org)

Humble Documentation: [docs.ros.org](https://docs.ros.org)

- See the Tutorials section for helpful ROS2 examples

Control Documentation: [control.ros.org/humble/](https://control.ros.org/humble/)

Tutorials:

[roboticsbackend.com/category/ros2/](https://roboticsbackend.com/category/ros2/)

[app.theconstruct.ai/courses/ros2-basics-in-5-days-humble-c-133/](https://app.theconstruct.ai/courses/ros2-basics-in-5-days-humble-c-133/)

Note: this course is robotics centric, but the principles discussed are relevant for MODAQ2 development

### Ubuntu

Installing Ubuntu 22.04 (x86\_64, AMD64): [ubuntu.com/tutorials/install-ubuntu-desktop](https://ubuntu.com/tutorials/install-ubuntu-desktop). Download install image [here](#)

Ubuntu 24.04 (Arm64) install image.

Foxglove GUI software: [docs.foxglove.dev/docs/introduction](https://docs.foxglove.dev/docs/introduction)

### C++

Reference: [en.cppreference.com/w/cpp](https://en.cppreference.com/w/cpp)

Tutorials:

- thenewboston Youtube channel: [youtube.com/playlist?list=PLAE85DE8440AA6B83](https://youtube.com/playlist?list=PLAE85DE8440AA6B83)
- C++ for robotics course: [app.theconstruct.ai/courses/c-for-robotics-59/](https://app.theconstruct.ai/courses/c-for-robotics-59/)

### Git and Github

- Managing personal access tokens: [managing-your-personal-access-tokens](#)
- Forking a github repo: [fork-a-repo](#)
- Using git submodules for modularity in versioning: [working-with-submodules](#)
- Creating a pull request with a forked repo: [creating-a-pull-request-from-a-fork](#)